HOMEWORK PROBLEM-3

QUES 1: Write a C program to store n elements in an array using dynamic memory allocation and find the largest number.

CODE:

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
   int n;
   printf("enter a number: ");
   scanf("%d",&n);
   int *arr=(int *)malloc(n*(sizeof(int)));
   printf("enter the elements: ");
   for(int i=0;i<n;i++)
   {
      scanf("%d",&arr[i]);
   }
   int largest=arr[0];
   for(int i=1;i<n;i++)
   {
      if(arr[i]>arr[0])
      {
         largest=arr[i];
      }
   }
   printf("\n the largest element is: %d",largest);
   free(arr);
   return 0;
}
```

QUES 2: Print an array in reverse order using pointer.

```
#include <stdio.h>
int main()
{
   int n;
   printf("enter a number: ");
   scanf("%d", &n);
   int arr[n];
   printf("enter the elements: ");
   for (int i = 0; i < n; i++)
   {
      scanf("%d", &arr[i]);
   }
   int *p;
   p=arr+n-1;
   printf("the reversed order of the elements: ");
   for(int i=0;i<n;i++)
   {
      printf("%d ",*p);
      p--;
   }
   return 0;
}
```

QUES 3: Write a C program to store information of the students using structure. Sort them by name.

CODE :

```
#include <stdio.h>
```

```
#include <string.h>
struct student
{
   int roll_no;
   char name[25];
};
int main()
{
   int n;
   printf("enter the number of the students: \n");
   scanf("%d", &n);
   struct student s[n];
   printf("enter the details of the students: \n");
   for (int i = 0; i < n; i++)
   {
      printf("enter roll no of student: %d\n", i + 1);
      scanf("%d", &s[i].roll_no);
      printf("enter name of student: %d\n", i + 1);
      scanf("%s", s[i].name);
   }
   for (int i = 0; i < n - 1; i++)
   {
      for (int j = i + 1; j < n; j++)
      {
         if (strcmp(s[i].name, s[j].name) > 0)
         {
            struct student temp;
            temp = s[i];
            s[i] = s[j];
```

```
            s[j] = temp;
          }
        }
    }
    printf("the sorted names and there roll numbers are: ");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", s[i].roll_no);
        printf("%s\n", s[i].name);
    }
    return 0;
}
```

QUES 4: Write a C program to add ‘n’ distances given in inch-feet system using structures

CODE:

```
#include <stdio.h>
struct distance
{
    int feet;
    int inch;
};
int main()
{
    int n;
    printf("enter number of distances: ");
    scanf("%d", &n);
    struct distance d[n];
    int total_feet = 0;
```

```
    int total_inch = 0;
    int extra=0,rem=0;
    for (int i = 0; i < n; i++)
    {
        printf("\nenter distance in feet: ");
        scanf("%d", &d[i].feet);
        printf("enter distance in inch: ");
        scanf("%d", &d[i].inch);
        total_feet += d[i].feet;
        total_inch += d[i].inch;
    }
    if(total_inch>=12)
    {
        extra=total_inch/12;
        total_feet+=extra;
        rem=total_inch%12;

        printf("the distance is: %d.%d",total_feet,rem);
    }
    else
    {
        printf("the distance is: %d.%d feet",total_feet,total_inch);
    }

    return 0;
}
```

QUES 5: Write a c program to add two complex numbers by passing structure for a function.

CODE:

```
#include <stdio.h>
struct complex
{
   int real, img;
};
struct complex add(struct complex a,struct complex b)
{
   struct complex temp;
   temp.real=a.real+b.real;
   temp.img=a.img+b.img;
   return temp;
}
int main()
{
   struct complex c[2];
   for (int i = 0; i < 2; i++)
   {
      printf("enter the real part: ");
      scanf("%d", &c[i].real);
      printf("enter the imaginary part: ");
      scanf("%d", &c[i].img);
   }
   struct complex result=add(c[0],c[1]);
   printf("Sum= %d+%di",result.real,result.img);
   return 0;
}
```

QUES 6: Write a C program to maintain inventory management system using structure.

CODE:

```
#include <stdio.h>
struct products
{
   int id;
   char name[25];
   float price;
   int quantity;
};
void printchoice()
{
   printf("1.Add products\n2.Display product details\n3.Search for a product\n4.calculate total inventory value\n5.Exit");
}
void addproduct(struct products p[], int n)
{
   for (int i = 0; i < n; i++)
   {
      printf("\nenter id of product: %d", i + 1);
      scanf("%d", &p[i].id);
      printf("\nenter name of product: %d", i + 1);
      scanf("%s", p[i].name);
      printf("\nenter price of product: %d", i + 1);
      scanf("%f", &p[i].price);
      printf("\nenter quantity of product: %d", i + 1);
      scanf("%d", &p[i].quantity);
   }
}
void display(struct products p[], int n)
```

```
{
   for (int i = 0; i < n; i++)
   {
      printf("\nthe id of the product %d is: %d", i + 1, p[i].id);
      printf("\nthe name of the product %d is: %s", i + 1, p[i].name);
      printf("\nthe price of the product %d is: %f", i + 1, p[i].price);
      printf("\nthe quantity of the product %d is: %d", i + 1, p[i].quantity);
   }
}
void search(struct products p[], int n)
{
   int id;
   printf("\nenter the product's id to search: ");
   scanf("%d", &id);
   for (int i = 0; i < n; i++)
   {
      if (id == p[i].id)
      {
         printf("\nid: %d", p[i].id);
         printf("\nname: %s", p[i].name);
         printf("\nprice: %f", p[i].price);
         printf("\nquantity: %d", p[i].quantity);
      }
   }
}
void calculate(struct products p[], int n)
{
   float total = 0;
   for (int i = 0; i < n; i++)
```

```
    {
        total += p[i].price * p[i].quantity;
    }
    printf("\ntotal inventory value is %f: ", total);
}
int main()
{
    int n;
    printf("enter the number of the products: ");
    scanf("%d", &n);
    struct products p[n];
    int choice;
    do
    {
        printchoice();
        printf("\nenter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
        case 1:
            addproduct(p, n);
            break;
        case 2:
            display(p, n);
            break;
        case 3:
            search(p, n);
            break;
        case 4:
```

```
            calculate(p, n);
            break;
        case 5:
            printf("exiting");
            break;

        default:
            printf("invalid input");
            break;
        }
    } while (choice != 5);

    return 0;
}
```

QUES 7: Write a C program for decimal to binary, octal and hexadecimal conversion.

CODE:

```
#include <stdio.h>
int main()
{
    int n, temp, i = 0, j = 0;
    printf("enter the number: ");
    scanf("%d", &n);
    int binary[32], octal[32], hexa[32];
    temp = n;
    while (temp != 0)
    {
        binary[i] = temp % 2;
        temp = temp / 2;
```

```
    i++;
}
printf("the binary form of the number is: ");
for (int j = i-1; j >= 0; j--)
{
    printf("%d", binary[j]);
}
printf("\nthe octal form of the number is: ");
temp=n;
i=0;
while (temp != 0)
{
    octal[i] = temp % 8;
    temp = temp / 8;
    i++;
}
for (int j = i-1; j >= 0; j--)
{
    printf("%d", octal[j]);
}
printf("\nthe hexadecimal form of the number is: ");
temp=n;
i=0;
while (temp != 0)
{
    hexa[i] = temp % 16;
    temp = temp / 16;
    i++;
}
```

```
    for (int j = i-1; j >= 0; j--)
    {
        printf("%d", hexa[j]);
    }

    return 0;
}
```

QUES 8: Write a C program to calculate difference between two time period.

CODE :

```
#include <stdio.h>
struct time {
    int hour;
    int minute;
    int second;
};

int main()
{
    struct time t1, t2, diff;

    printf("Enter first time (HH MM SS): ");
    scanf("%d %d %d", &t1.hour, &t1.minute, &t1.second);

    printf("Enter second time (HH MM SS): ");
    scanf("%d %d %d", &t2.hour, &t2.minute, &t2.second);

    if (t2.hour < t1.hour ||
```

```
    (t2.hour == t1.hour && t2.minute < t1.minute) ||
    (t2.hour == t1.hour && t2.minute == t1.minute &&
     t2.second < t1.second))
{
    struct time temp = t1;
    t1 = t2;
    t2 = temp;
}

if (t2.second < t1.second)
{
    t2.second += 60;
    t2.minute--;
}
diff.second = t2.second - t1.second;

if (t2.minute < t1.minute)
{
    t2.minute += 60;
    t2.hour--;
}
diff.minute = t2.minute - t1.minute;

diff.hour = t2.hour - t1.hour;

printf("Difference = %d hours %d minutes %d seconds\n",
       diff.hour, diff.minute, diff.second);

return 0;
```

```
}
```

QUES 9: Write a C program to implement an employee management system using structures.

CODE:

```
#include <stdio.h>
struct employee
{
   int emp_id;
   char emp_name[25];
   int age;
   float salary;
};
void printchoice()
{
   printf("1.Add employee\n2.Display Details\n3.Search\n4.Exit");
}
void addemp(struct employee a[], int n)
{
   printf("\n enter the employee's details\n");
   for (int i = 0; i < n; i++)
   {
      printf("enter id of employee %d", i + 1);
      scanf("%d", &a[i].emp_id);
      printf("enter name of employee %d", i + 1);
      scanf("%s", a[i].emp_name);
      printf("enter age of employee %d", i + 1);
      scanf("%d", &a[i].age);
      printf("enter salary of employee %d", i + 1);
```

```
        scanf("%f", &a[i].salary);
    }
}
void display(struct employee a[], int n)
{
    printf("\nAll the details of the employees");
    for (int i = 0; i < n; i++)
    {
        printf("\nID: %d", a[i].emp_id);
        printf("\nNAME: %s", a[i].emp_name);
        printf("\nAGE: %d", a[i].age);
        printf("\nSALARY: %f", a[i].salary);
    }
}
void search(struct employee a[], int n)
{
    int id;
    printf("enter the id to search an employee: ");
    scanf("%d", &id);
    for (int i = 0; i < n; i++)
    {
        if (id == a[i].emp_id)
        {
            printf("\nID: %d", a[i].emp_id);
            printf("\nNAME: %s", a[i].emp_name);
            printf("\nAGE: %d", a[i].age);
            printf("\nSALARY: %f", a[i].salary);
        }
        else
```

```
        {
            printf("employee not found");
        }
    }
}
int main()
{
    int n;
    printf("enter number of employee: ");
    scanf("%d", &n);
    struct employee e[n];
    int choice;
    do
    {
        printchoice();
        printf("\nenter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
        case 1:
            addemp(e, n);
            break;
        case 2:
            display(e, n);
            break;
        case 3:
            search(e, n);
            break;
        case 4:
```

```
            printf("exiting");
            break;
        default:
            printf("invalid choice");
            break;
        }

    } while (choice != 4);
    return 0;
}
```

QUES 10: Write a C program to calculate area and perimeters of a rectangle and triangle using function pointers.

CODE:

```
#include<stdio.h>
float arearec(float a, float b)
{
    return a*b;
}
float perirec(float x, float y)
{
    return 2*(x+y);
}
float areatri(float a, float b)
{
    return 0.5*(a*b);
}
float peritri(float x)
{
```

```
    return 3*x;
}
int main()
{
    float l,b,h,s;
    printf("enter the length of the rectangle: ");
    scanf("%f",&l);
    printf("enter the breadth of the rectangle: ");
    scanf("%f",&b);
    float (*p)(float,float);
    p=arearec;
    printf("\nthe area of the rectangle is: %f",p(l,b));
    p=perirec;
    printf("\nthe perimeter of the rectangle is: %f",p(l,b));
    printf("\nenter the heigth of the triangle: ");
    scanf("%f",&h);
    printf("\nenter the side of the triangle: ");
    scanf("%f",&s);
    p=areatri;
    printf("\nthe area of the triangle is: %f",p(h,s));
    float (*q)(float);
    q=peritri;
    printf("\n the perimeter of the rectangle is: %f",q(s));
    return 0;
}
```